



Linux Fundamentals

0%



Section 29 / 30

Solaris

Solaris is a Unix-based operating system developed by Sun Microsystems (later acquired by Oracle Corporation) in the 1990s. It is known for its robustness, scalability, and support for high-end hardware and software systems. Solaris is widely used in enterprise environments for mission-critical applications, such as database management, cloud computing, and virtualization. For example, it includes a built-in hypervisor called **Oracle VM Server for SPARC**, which allows multiple virtual machines to run on a single physical server. Overall, it is designed to handle large amounts of data and provide reliable and secure services to users and is often used in enterprise environments where security, performance, and stability are key requirements.

The goal of Solaris is to provide a highly stable, secure, and scalable platform for enterprise computing. It has built-in features for high availability, fault tolerance, and system management, making it ideal for mission-critical applications. It is widely used in the banking, finance, and government sectors, where security, reliability, and performance are paramount. It is also used in large-scale data centers, cloud computing environments, and virtualization platforms. Companies such as Amazon, IBM, and Dell use Solaris in their products and services, highlighting its importance in the industry.

Linux Distributions vs Solaris

Solaris and Linux distributions are two types of operating systems that differ significantly. Firstly, Solaris is a proprietary operating system owned and developed by Oracle Corporation, and its source code is not available to the general public. In contrast, most Linux distributions are open-source, meaning that their source code is available for anyone to modify and use. Additionally, Linux distributions commonly use the Zettabyte File System ([ZFS](#)), which is a highly advanced file system that offers features such as data compression, snapshots, and high scalability. On the other hand, Solaris uses a Service Management Facility ([SMF](#)), which is a highly advanced service management framework that provides better reliability and availability for system services.

Directory	Description
/	The root directory contains all other directories and files in the file system.
/bin	It contains essential system binaries that are required for booting and basic system operations.
/boot	The boot directory contains boot-related files such as boot loader and kernel images.
/dev	The dev directory contains device files that represent physical and logical devices attached to the system.
/etc	The etc directory contains system configuration files, such as system startup scripts and user authentication data.

Directory	Description
<code>/home</code>	Users' home directories.
<code>/kernel</code>	This directory contains kernel modules and other kernel-related files.
<code>/lib</code>	Directory for libraries required by the binaries in <code>/bin</code> and <code>/sbin</code> directories.
<code>/lost+found</code>	This directory is used by the file system consistency check and repair tool to store recovered files.
<code>/mnt</code>	Directory for mounting file systems temporarily.
<code>/opt</code>	This directory contains optional software packages that are installed on the system.
<code>/proc</code>	The <code>proc</code> directory provides a view into the system's process and kernel status as files.
<code>/sbin</code>	This directory contains system binaries required for system administration tasks.
<code>/tmp</code>	Temporary files created by the system and applications are stored in this directory.
<code>/usr</code>	The <code>usr</code> directory contains system-wide read-only data and programs, such as documentation, libraries, and executables.

Directory	Description
<code>/var</code>	This directory contains variable data files, such as system logs, mail spools, and printer spools.

Solaris has a number of unique features that set it apart from other operating systems. One of its key strengths is its support for high-end hardware and software systems. It is designed to work with large-scale data centers and complex network infrastructures, and it can handle large amounts of data without any performance issues.

In terms of package management, Solaris uses the Image Packaging System (`IPS`) package manager, which provides a powerful and flexible way to manage packages and updates. Solaris also provides advanced security features, such as Role-Based Access Control (`RBAC`) and mandatory access controls, which are not available in all Linux distributions.

Differences

Let's dive deeper into the differences between Solaris and Linux distributions. One of the most important differences is that the source code is not open source and is only known in closed circles. This means that unlike Ubuntu or many other distributions, the source code cannot be viewed and analyzed by the public. In summary, the main differences can be grouped into the following categories:

- Filesystem
- Process management

- Package management
- Kernel and Hardware support
- System monitoring
- Security

To better understand the differences, let's take a look at a few examples and commands.

System Information

On Ubuntu, we use the `uname` command to display information about the system, such as the kernel name, hostname, and operating system. This might look like this:

```
athane@htb[/htb]$ uname -a
```

```
Linux ubuntu 5.4.0-1045 #48-Ubuntu SMP Fri Jan 15 10:47:29 UTC 2021 x86_64 x86_64  
x86_64 GNU/Linux
```

shellsession

On the other hand, in Solaris, the `showrev` command can be used to display system information, including the version of Solaris, hardware type, and patch level. Here is an example output:

```
$ showrev -a
```

```
Hostname: solaris  
Kernel architecture: sun4u  
OS version: Solaris 10 8/07 s10s_u4wos_12b SPARC  
Application architecture: sparc
```

shellsession

```
Hardware provider: Sun_Microsystems  
Domain: sun.com  
Kernel version: SunOS 5.10 Generic_139555-08
```

The main difference between the two commands is that `showrev` provides more detailed information about the Solaris system, such as the patch level and hardware provider, while `uname` only provides basic information about the Linux system.

Installing Packages

On Ubuntu, the `apt-get` command is used to install packages. This could look like the following:

```
athane@htb[/htb]$ sudo apt-get install apache2
```

shellsession

However, in Solaris, we need to use `pkgadd` to install packages like `SUNWapchr`.

```
$ pkgadd -d SUNWapchr
```

shellsession

The main difference between the two commands is the syntax, and the package manager used. Ubuntu uses the Advanced Packaging Tool (APT) to manage packages, while Solaris uses the Solaris Package Manager (SPM). Also, note that we do not use `sudo` in this case. This is because Solaris used the `RBAC` privilege management tool, which allowed the assignment of granular permissions to users. However, `sudo` has been supported since Solaris 11.

Permission Management

On Linux systems like Ubuntu but also on Solaris, the `chmod` command is used to change the permissions of files and directories. Here is an example command to give read, write, and execute permissions to the owner of the file:

```
athane@htb[/htb]$ chmod 700 filename
```

shellsession

To find files with specific permissions in Ubuntu, we use the `find` command. Let us take a look at an example of a file with the SUID bit set:

```
athane@htb[/htb]$ find / -perm 4000
```

shellsession

To find files with specific permissions, like with the SUID bit set on Solaris, we can use the `find` command, too, but with a small adjustment.

```
$ find / -perm -4000
```

shellsession

The main difference between these two commands is the use of the `-` before the permission value in the Solaris command. This is because Solaris uses a different permission system than Linux.

NFS in Solaris

Solaris has its own implementation of NFS, which is slightly different from Linux distributions like Ubuntu. In Solaris, the NFS server can be configured using the `share` command, which is used to share a directory over the network, and it also allows us to specify various options such as read/write permissions, access restrictions, and more. To share a directory over NFS in Solaris, we can use the following command:

```
$ share -F nfs -o rw /export/home
```

shellsession

This command shares the `/export/home` directory with read and writes permissions over NFS. An NFS client can mount the NFS file system using the `mount` command, the same way as with Ubuntu. To mount an NFS file system in Solaris, we need to specify the server name and the path to the shared directory. For example, to mount an NFS share from a server with the IP address `10.129.15.122` and the shared directory `/nfs_share`, we use the following command:

```
athane@htb[/htb]$ mount -F nfs 10.129.15.122:/nfs_share /mnt/local
```

shellsession

In Solaris, the configuration for NFS is stored in the `/etc/dfs/dfstab` file. This file contains entries for each shared directory, along with the various options for NFS sharing.

```
# cat /etc/dfs/dfstab

share -F nfs -o rw /export/home
```

shellsession

Process Mapping

Process mapping is an essential aspect of system administration and troubleshooting. The `lsof` command is a powerful utility that lists all the files opened by a process, including network sockets and other file descriptors that we can use in Debian distributions like Ubuntu. We can use `lsof` to list all the files opened by a process. For example, to list all the files opened by the Apache web server process, we can use the following command:

```
athane@htb[/htb]$ sudo lsof -c apache2
```

shellsession

In Solaris, the `pfiles` command can be used to list all the files opened by a process. For example, to list all the files opened by the Apache web server process, we can use the following command:

```
$ pfiles `pgrep httpd`
```

shellsession

This command lists all the files opened by the Apache web server process. The output of the `pfiles` command is similar to the output of the `lsof` command and provides information about the type of file descriptor, the file descriptor number, and the file name.

Executable Access

In Solaris, `truss` is used, which is a highly useful utility for developers and system administrators who need to debug complex software issues on the Solaris operating system. By tracing the system calls made by a process, `truss` can help identify the source of errors, performance

issues, and other problems but can also reveal some sensitive information that may arise during application development or system maintenance. The utility can also provide detailed information about system calls, including the arguments passed to them and their return values, allowing users to better understand the behavior of their applications and the underlying operating system.

Strace is an alternative to **truss** but for Ubuntu, and it is an essential tool for system administrators and developers alike, helping them diagnose and troubleshoot issues in real-time. It enables users to analyze the interactions between the operating system and applications running on it, which is especially useful in highly complex and mission-critical environments. With **truss**, users can quickly identify and isolate issues related to application performance, network connectivity, and system resource utilization, among others.

For example, to trace the system calls made by the Apache web server process, we can use the following command:

```
athane@htb[/htb]$ sudo strace -p `pgrep apache2`
```

shellsession

Here's an example of how to use **truss** to trace the system calls made by the **ls** command in Solaris:

```
$ truss ls
```

```
execve("/usr/bin/ls", 0xFFBFFDC4, 0xFFBFFDC8)  argc = 1  
...SNIP...
```

shellsession

The output is similar to `strace`, but the format is slightly different. One difference between `strace` and `truss` is that `truss` can also trace the signals sent to a process, while `strace` cannot. Another difference is that `truss` has the ability to trace the system calls made by child processes, while `strace` can only trace the system calls made by the process specified on the command line.



29 / 30 Sections

[Mark Complete & Next](#)